UNIDAD 5:

Formularios, ficheros y formatos de intercambio de datos

Índice

1. Introducción a los formularios 2
1.1. Atributos action y method 2
1.2. Método GET 2
1.3. Método POST 3
1.4. Comparativa GET y POST 4
2. Acceso a formularios desde PHP 4
2.1. Acceso a diferentes tipos de elementos 6
2.2. Agrupando variables en un array 9
3. Procesamiento de formularios con MVC 10
3.1. El ciclo de un formulario 10
3.2. Estructura del proyecto 11
3.3. El controlador frontal y la autocarga 11
3.4. El modelo 12
3.5. El controlador 13
3.6. Las vistas 13
3.7. Qué ocurre al recargar la página 15
4. Validación y saneamiento de formularios 15
4.1. Opciones de filtrado y banderas (flags) 16
4.2. Filtros de validación más usados 16
4.3. Filtros de saneamiento 18
4.4. Escapar la salida: htmlspecialchars() 18
4.5. Validar y volver a mostrar el formulario en MVC 18

1. Introducción a los formularios

Los formularios son la forma más habitual de **enviar datos** a un servidor. El usuario interactúa con los controles en el navegador (HTML) y, al enviar el formulario, el servidor (PHP) procesa las variables y genera una respuesta dinámica.

Los formularios no forman parte de PHP, sino de HTML.

Hasta el momento solo hemos enviado información por la URL, de modo que los datos siempre eran visibles. Cuando usamos un formulario lo deseable es que los datos no se vean en la URL.

1.1. Atributos action y method

Al describir un formulario especificamos dos atributos fundamentales:

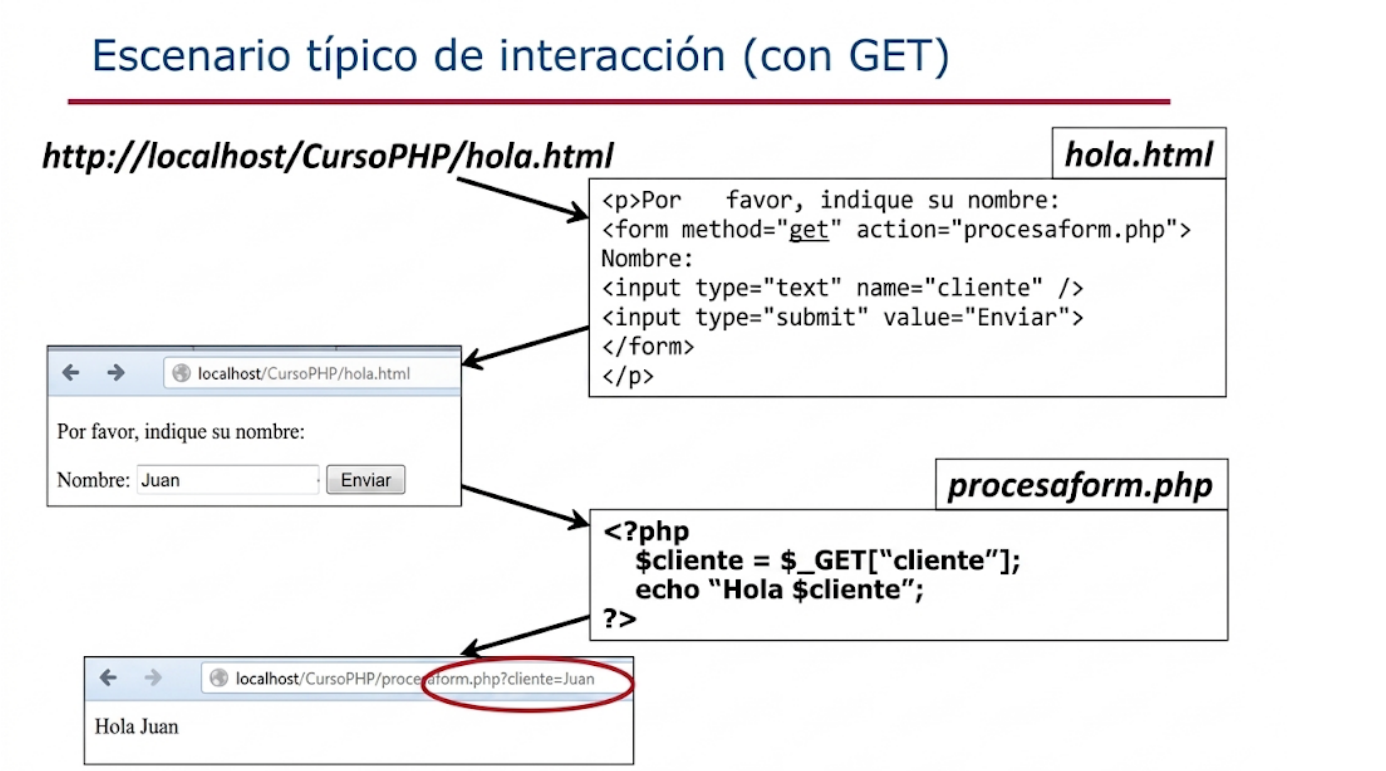
- `action`: define la ruta del script del servidor que procesará los datos del formulario.
- `method`: especifica el método HTTP que se usará para la petición. El tipo de envío influye en la forma en que PHP captura los datos. Generalmente se emplea **POST** para operaciones que modifican información o manejan datos sensibles, y **GET** para consultas o búsquedas.

1.2. Método GET

Los parámetros viajan incrustados en la URL con la estructura `fichero.php?param1=val1&param2=val2`. Ejemplo:

```
http://localhost/unidad5/saludo.php?nombre=Ana&apellido=Prieto
```

- Debemos evitar enviar información sensible a través de GET. **Jamás** debe usarse para contraseñas o datos personales.
- Debemos utilizarlo, en general, solo cuando proporcionamos al servidor información relativa al contenido que queremos recuperar (orden, página, cantidad, etc.).
- Es ideal para mejorar el SEO y permite guardar las búsquedas en los marcadores del navegador. Sin embargo, como el usuario puede modificar la URL, el código PHP debe estar preparado para gestionar parámetros alterados o maliciosos.

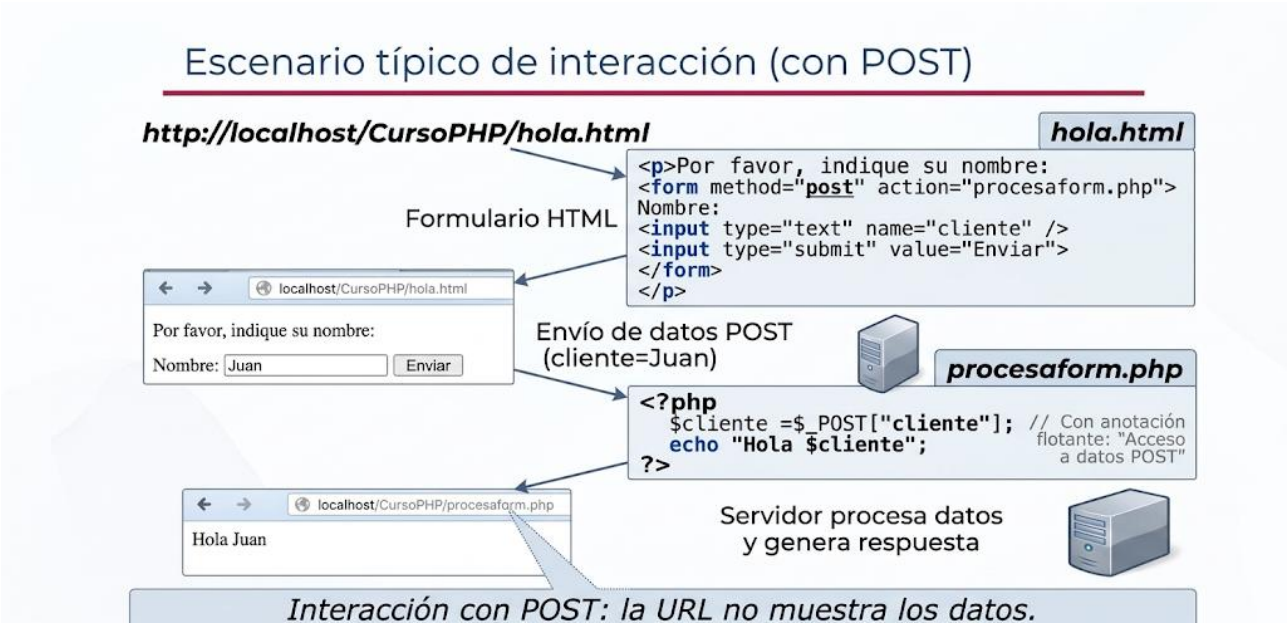


Interacción con GET: los datos aparecen en la URL.

1.3. Método POST

Los datos viajan en el **cuerpo** de la petición HTTP, por lo que no aparecen en la URL.

- Es el método adecuado para altas, modificaciones, inicios de sesión y envío de ficheros.
- Al recargar la página, el navegador pide confirmación para reenviar el formulario (el problema se resuelve con el patrón *Post/Redirect/Get* del apartado 3.3).
- Que los datos no se vean en la URL **no significa que estén cifrados**: para protegerlos hace falta HTTPS.



1.4. Comparativa GET y POST

Característica	GET	POST
Dónde viajan los datos	En la URL	En el cuerpo de la petición
Visibilidad	Visibles y guardados en el historial	No aparecen en la URL
Tamaño	Limitado por la longitud de la URL	Limitado por `post_max_size`
Marcadores y caché	Se pueden guardar y cachear	No se guardan en marcadores
Al recargar la página	Se repite la consulta sin avisar	El navegador pide confirmar el reenvío
Uso recomendado	Consultas, filtros, búsquedas, paginación	Altas, modificaciones, login, subida de ficheros
Variable de PHP	`$_GET`	`$_POST` (y `$_FILES` para ficheros)

2. Acceso a formularios desde PHP

Desde PHP obtenemos la información enviada por el formulario mediante las variables superglobales:

- **`$_GET`**: array con la información enviada a través de los parámetros de la URL. Tiene un elemento por cada argumento presente en la URL; el nombre del argumento es la clave.
- **`$_POST`**: contiene las variables enviadas en el cuerpo de la petición con el método POST. La clave de cada argumento es el atributo `name` del elemento correspondiente del formulario.
- **`$_REQUEST`**: agrupa `$_GET`, `$_POST` y, según la configuración (`request_order`), también `$_COOKIE`.

> **Atención**
> **Evita `$_REQUEST`.** Es más seguro y claro ser explícito con `$_POST` y `$_GET`. Así no se confunde de dónde procede un dato ni se permite, por ejemplo, que el valor de una cookie sobrescriba un dato enviado por POST.

Si intentamos leer una clave de un array que no se ha enviado, PHP 8 emite un aviso *Warning: Undefined array key* (en PHP 7 era un *Notice*). La forma idónea y más limpia de dar un valor por defecto es el **operador `??`**:

```php
// Si no llega el parámetro se asigna un valor por defecto, sin avisos
$nombre   = $_POST['nombre'] ?? '';
$apellido = $_POST['apellido'] ?? '';
```

Dentro del elemento form se especifican los campos que rellenará el usuario, así como los botones de envío y de reseteo. Ejemplo de formulario HTML:

```
<form method="POST" action="nuevoAmigo.php">
    <label for="nombre">Nombre</label>
    <input type="text" id="nombre" name="nombre">

    <label for="apellido">Apellido</label>
    <input type="text" id="apellido" name="apellido">

    <input type="submit" value="Enviar datos">
</form>
```

Para que un control envíe información es necesario que:

- el control esté incluido en un formulario (`<form>`);
- el formulario tenga establecido el atributo `action` (si se omite, se envía a la misma URL);
- el control tenga establecido el atributo `name`;
- el formulario contenga un botón de tipo `submit`.

> **Nota**
> El atributo `name` puede contener cualquier carácter (números, acentos, guiones…), pero PHP sustituye los **espacios y los puntos** por guiones bajos (_) al construir `$_POST` y `$_GET`. Un campo `name="mi nombre"` se lee como `$_POST['mi_nombre']`.

El fichero que recibe la información, `nuevoAmigo.php`, podría ser el siguiente:

```
<?php declare(strict_types=1);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nombre   = trim($_POST['nombre'] ?? '');
    $apellido = trim($_POST['apellido'] ?? '');

    if ($nombre !== '' && $apellido !== '') {
        echo '<h1>Datos recibidos:</h1>';
        echo '<h2>Nombre: ' . htmlspecialchars($nombre) . '</h2>';
        echo '<h2>Apellidos: ' . htmlspecialchars($apellido) . '</h2>';
    }
}
```

> **Atención**
> Todo dato que procede del usuario y se muestra en una página HTML debe pasar por `htmlspecialchars()`. Si se imprime `$_POST['nombre']` tal cual, un usuario podría introducir código JavaScript (ataque **XSS**). Se explica con detalle en el apartado 4.4.

En realidad podemos incluir el código PHP dentro del propio HTML para mostrar los datos. Así quedaría el ejemplo anterior:

```
<?php
declare(strict_types=1);

// Recoger datos
$nombre   = htmlspecialchars($_POST['nombre'] ?? '', ENT_QUOTES, 'UTF-8');
$apellido = htmlspecialchars($_POST['apellido'] ?? '', ENT_QUOTES, 'UTF-8');
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Datos recibidos</title>
</head>
<body>
    <h1>Datos recibidos</h1>
    <p><strong>Nombre:</strong> <?= $nombre ?: 'No especificado' ?></p>
    <p><strong>Apellidos:</strong> <?= $apellido ?: 'No especificado' ?></p>
</body>
</html>
```

2.1. Acceso a diferentes tipos de elementos

Cada tipo de control envía su información de una forma distinta. Este resumen indica qué se recibe y cómo se lee en PHP:

Control	Qué se recibe	Lectura en PHP
`text`, `textarea`, `hidden`, `password`, `select` simple	Una cadena con el valor	`$_POST['x'] ?? ''`
`radio`	El `value` del seleccionado. Si no hay ninguno, no se envía nada	`$_POST['x'] ?? ''`
`checkbox` único	Su `value` (`on` si no tiene) solo si está marcado	`isset($_POST['x'])`
`checkbox` en grupo, `select multiple`	Un array con los valores marcados (`name="x[]"`)	`$_POST['x'] ?? []`
`submit`, `button`	El `value` del botón pulsado, si tiene `name`	`$_POST['x'] ?? ''`
`file`	No va en `$_POST`: sus datos están en `$_FILES`	Ver apartado 8

Text, Textarea, Hidden y Password

Se accede mediante la clave correspondiente al atributo `name` del control HTML.

```
<form method="POST" action="procesar.php">
    Contraseña: <input type="password" name="clave">
    <input type="submit" value="Enviar">
</form>
```

```php
<?php
$clave = $_POST['clave'] ?? '';
```

> **Atención**
> Un campo `hidden` **no es seguro**: el usuario puede modificar su valor con las herramientas del navegador. Nunca confíes en él para precios, identificadores de permisos, etc.
>
> Las contraseñas no se guardan nunca en claro: se almacenan con `password_hash()` y se comprueban con `password_verify()`.

Para mostrar el contenido de un `textarea` respetando los saltos de línea se combina `nl2br(htmlspecialchars($texto))`.

Radio

Envían únicamente el valor del botón seleccionado por el usuario.

```php
<form method="POST" action="tipoRadio.php">
    <label>Sexo:</label>
    <input type="radio" name="sexo" value="M" checked>Mujer
    <input type="radio" name="sexo" value="H">Hombre
    <input type="submit" value="enviar">
</form>
```

```php
<?php
$sexo = $_POST['sexo'] ?? '';   // vacío si el usuario no seleccionó ninguno
```

Checkbox

Para recuperar **varios valores** de un grupo de casillas (o de una lista de selección múltiple), el atributo `name` debe terminar obligatoriamente con corchetes `[]`. Esto le indica a PHP que debe empaquetar las selecciones en un array indexado.

```php
<form method="POST" action="tipoCheckbox.php">
    <input type="checkbox" name="extras[]" value="garaje" checked>Garaje
    <input type="checkbox" name="extras[]" value="piscina">Piscina
    <input type="checkbox" name="extras[]" value="jardin">Jardín
    <input type="submit" value="enviar">
</form>
```

`$_GET` y `$_POST` contienen en este caso un array con los valores recogidos, y por tanto hay que recorrerlo como tal. Las casillas sin marcar no se envían, de modo que si no hay ninguna marcada la clave no existe: por eso se usa `?? []`.

```php
$extras = $_POST['extras'] ?? [];

foreach ($extras as $extra) {
    echo htmlspecialchars($extra) . '<br>';
}
```

Select

Debemos distinguir entre la lista de selección simple y la múltiple.

Select simple

```
<form method="POST" action="tipoSelectSimple.php">
    <select name="color">
        <option value="rojo">rojo</option>
        <option value="verde" selected>verde</option>
        <option value="azul">azul</option>
    </select>
    <input type="submit" value="enviar">
</form>
```

Los datos se obtienen igual que en el tipo Text:

```
$color = $_POST['color'] ?? '';
```

Select múltiple

Hay que tener cuidado: el nombre debe terminar en `[]` y los datos llegan en un array.

```
<form method="POST" action="tipoSelectMultiple.php">
    <select multiple size="3" name="idiomas[]">
        <option value="francés">Francés</option>
        <option value="inglés" selected>Inglés</option>
        <option value="alemán">Alemán</option>
    </select>
    <input type="submit" value="enviar">
</form>
```

```
$idiomas = $_POST['idiomas'] ?? [];

foreach ($idiomas as $idioma) {
    echo htmlspecialchars($idioma) . '<br>';
}
```

Submit (varios botones de envío)

Aunque no es lo más habitual, es válido tener más de un botón `submit` en un mismo formulario. Para saber en el servidor cuál ha pulsado el usuario, todos los botones deben compartir el mismo `name` y tener un `value` diferente.

```
<form method="POST" action="tipoSubmit.php">
    <p>¿Blanco o negro?</p>
    <input type="submit" name="respuesta" value="Negro">
    <input type="submit" name="respuesta" value="Blanco">
</form>
```

```
<?php
$respuesta = $_POST['respuesta'] ?? 'Ninguno';
echo 'Has seleccionado: ' . htmlspecialchars($respuesta);
```

Button

Se puede lograr el mismo comportamiento con la etiqueta `<button>`. Hay que añadir `type="submit"` (porque en algunos navegadores el tipo por defecto puede variar) y mantener el mismo `name` con distintos `value`.

```
<form method="POST" action="tipoButton.php">
    <p>¿Qué prefieres para el diseño de la interfaz?</p>
    <button type="submit" name="interfaz" value="oscuro">Modo oscuro</button>
    <button type="submit" name="interfaz" value="claro">Modo claro</button>
</form>
```

```
<?php
$interfaz = $_POST['interfaz'] ?? 'no_definido';
echo 'El usuario ha preferido el estilo: ' . htmlspecialchars($interfaz);
```

File

Este elemento se estudia con detalle en el apartado 8 («Envío de archivos al servidor»).

2.2. Agrupando variables en un array

Podemos agrupar los nombres de los controles con una sintaxis de array asociativo directamente en el atributo `name` de HTML. Resulta muy útil para estructurar la información, por ejemplo cuando trabajamos con tablas de bases de datos. En realidad ya lo hemos usado con los grupos de checkbox.

```
<label for="nombre">Nombre</label>
<input type="text" id="nombre" name="contacto[nombre]">

<label for="apellidos">Apellidos</label>
<input type="text" id="apellidos" name="contacto[apellidos]">
```

En PHP, la superglobal genera automáticamente un array multidimensional:

```
<?php
$contacto  = $_POST['contacto'] ?? [];
$nombre    = $contacto['nombre'] ?? '';
$apellidos = $contacto['apellidos'] ?? '';

echo 'Nuevo contacto: ' . htmlspecialchars($nombre) . ' ' . htmlspecialchars($apellidos);
```

3. Procesamiento de formularios con MVC

En una aplicación MVC los formularios no se procesan en un script suelto: el trabajo se reparte entre capas con responsabilidades distintas. El ciclo del formulario es el mismo que en cualquier aplicación web; lo que cambia es **dónde vive cada pieza**. En este apartado se construye, paso a paso, un formulario de contacto con los campos : nombre, apellidos, correo electrónico, métodos de pago y aceptación de los términos y condiciones.

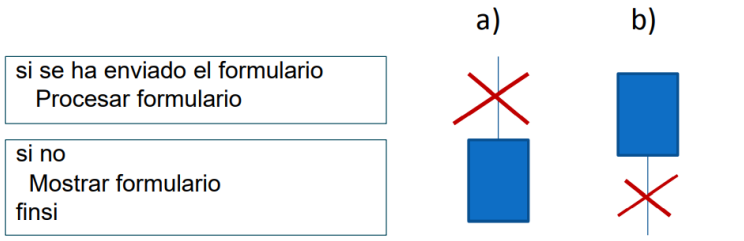
> **Nota**
> Se usa una estructura MVC mínima, sin framework y sin Composer: las clases se cargan con el autoloader que ya creamos. Todavía no hemos visto ficheros ni bases de datos, así que la aplicación **no guarda nada**, y tampoco **valida** los datos: se limita a recibirlos y mostrarlos.

3.1. El ciclo de un formulario

Un formulario no es una única petición, sino un pequeño ciclo con dos pasos:

- `?controller=contacto&action=formulario`: el usuario pide la página y se **muestra** el formulario vacío (con VISA preseleccionada).
- `?controller=contacto&action=procesar`: el usuario envía los datos por POST y se **procesan**: el controlador los recoge, crea el modelo y muestra la vista de **resultado** con los datos recibidos. No se valida ni se guarda nada.

Fíjate en que la **misma página** (`index.php`) sirve para mostrar y para procesar: lo que decide qué se ejecuta son los parámetros `controller` y `action` de la URL. El formulario envía sus datos por POST, pero esos dos parámetros siguen viajando por GET en la URL de destino.



- La 1ª vez que se carga la página se muestra el formulario (a)
- La 2ª vez se procesa el formulario (b)

Primera carga (a): se muestra el formulario. Segunda carga (b): se procesa.

3.2. Estructura del proyecto

```
proyecto/
├── public/                      ← única carpeta accesible desde el navegador
│   └── index.php                ← único punto de entrada
├── config/
│   └── config.php               ← constantes CONTROLLER_DEFAULT y ACTION_DEFAULT
├── autoloader.php               ← carga las clases desde src/
├── src/                         ← código PHP de la aplicación
│   ├── Controllers/
│   │   ├── FrontController.php  ← controlador frontal
│   │   ├── ErrorController.php  ← muestra el error 404
│   │   └── ContactoController.php
│   ├── Lib/                     ← clases de apoyo (también puede llamarse Core/)
│   │   ├── Pages.php            ← carga las plantillas (vistas)
│   │
│   └── Models/
│       └── Contacto.php         ← datos y validación
└── Views/                       ← plantillas
    ├── Layout/
    │   ├── header.php           ← cabecera común (hasta abrir <main>)
    │   └── footer.php           ← pie común (cierra <main>, <body> y <html>)
    └── contacto/
        ├── formulario.php
        └── resultado.php
```

> **Nota**
> **Lib o Core.** La carpeta de clases de apoyo se llama `Lib` en estos apuntes, pero en otros proyectos y frameworks la verás con el nombre `Core`. Lo importante es que el **namespace coincida con la ruta de la carpeta** dentro de `src/` (`Lib\Pages` → `src/Lib/Pages.php`; si la carpeta fuera `Core`, el namespace sería `Core\Pages`), porque es lo que usa el autoloader para encontrar las clases.

3.3. El controlador frontal y la autocarga

Todas las peticiones entran por `public/index.php`. Este fichero hace tres cosas: carga la **configuración** (las constantes con el controlador y la acción por defecto), activa el **autoloader** casero (para no escribir un `require` por cada clase) y llama a `FrontController::main()`. El fichero `helpers.php` se incluye aparte porque contiene una función, no una clase, y el autoloader solo carga clases.

Fichero: `config/config.php`

```php
<?php
declare(strict_types=1);

// Controlador y acción que se ejecutan cuando la URL no indica ninguno
const CONTROLLER_DEFAULT = 'contacto';
const ACTION_DEFAULT = 'formulario';
```

3.4. El modelo

`Contacto` representa los datos del formulario.Todos los parámetros tienen valor por defecto, de modo que `new Contacto()` crea un contacto vacío.

El método `fromArray()` crea el objeto a partir del array `contacto[...]` que envía el formulario (apartado 2.2). Quita los espacios sobrantes con `trim()` y usa `isset()` para la casilla de los términos, porque una casilla sin marcar no se envía (apartado 2.1).

Fichero: `src/Models/Contacto.php`

```php
<?php
declare(strict_types=1);

namespace Models;

class Contacto
{
    /** @param string[] $metodosPago */
    public function __construct(
        public string $nombre = '',
        public string $apellidos = '',
        public string $correo = '',
        public array $metodosPago = [],
        public bool $marcar = false,
    ) {}

    /** Crea un Contacto a partir del array contacto[...] del formulario */
    public static function fromArray(array $datos): self
    {
        return new self(
            nombre: trim($datos['nombre'] ?? ''),
            apellidos: trim($datos['apellidos'] ?? ''),
            correo: trim($datos['correo'] ?? ''),
            metodosPago: $datos['metodosPago'] ?? [],
            marcar: isset($datos['marcar']),
        );
    }
}
```

> **Atención**
> Los atributos `required` y `minlength` los comprueba únicamente el navegador, y se saltan con facilidad. La validación en el servidor (que el nombre tenga al menos 5 caracteres, que el correo sea válido, etc.) se verá en el apartado 4 y se aplicará a este formulario en el ejercicio 11.1.

> **Consejo**
> El modelo no lee `$_POST` ni escribe HTML. Por eso se puede reutilizar desde un formulario web, desde una API o desde un script de línea de comandos.

3.5. El controlador

El controlador tiene una acción por cada paso del ciclo. En `procesar()` se ve el recorrido: **recoger** los datos de `$_POST`, crear el **modelo** y mostrar el **resultado**. Las vistas se cargan con la clase `Pages`, que se crea en el constructor.

Fichero: `src/Controllers/ContactoController.php`

```php
<?php
declare(strict_types=1);

namespace Controllers;

use Lib\Pages;
use Models\Contacto;

class ContactoController
{
    private Pages $pagina;

    public function __construct()
    {
        $this->pagina = new Pages();
    }

    /** muestra el formulario */
    public function formulario(): void
    {
        $this->pagina->render('contacto/formulario');
    }

    /** Recoge los datos y muestra el resultado */

    public function procesar(): void
    {
        // Si se entra por GET no hay datos: se muestra el formulario
        if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
            $this->formulario();
            return;
        }

        // Recoger los datos
        // Recuerda que esto es peligroso y que tenemos que sanear y validad
        $contacto = Contacto::fromArray($_POST['contacto'] ?? []);

        // Mostrar el resultado (todavía no se valida ni se guarda nada)
        $this->pagina->render('contacto/resultado', ['contacto' => $contacto]);
    }
}
```

Qué hace y qué no hace un controlador:

- **Sí:** leer `$_POST` y `$_GET`; llamar al modelo; elegir la vista y pasarle los datos.
- **No:** escribir HTML ni contener las reglas de negocio (esas están en el modelo).

3.6. Las vistas

Las plantillas las carga la clase `Pages` (`src/Lib/Pages.php`). Su método `render()` convierte cada clave del array en una variable (`'contacto' => ...` se convierte en `$contacto`) y carga tres

ficheros de `Views/` seguidos: `Layout/header.php`, la plantilla pedida y `Layout/footer.php`. Así, la cabecera y el pie se escriben una sola vez y cada vista contiene únicamente su parte.

Fichero: `Views/contacto/formulario.php`

```
<h1>Nuevo contacto</h1>

<form method="POST" action="index.php?controller=contacto&amp;action=procesar">
<p>
    <label for="nombre">Nombre</label>
    <input type="text" id="nombre" name="contacto[nombre]"/>
</p>

<p>
    <label for="apellidos">Apellidos</label>
    <input type="text" id="apellidos" name="contacto[apellidos]" />
</p>

<p>
    <label for="correo">Correo electrónico</label>
    <input type="email" id="correo" name="contacto[correo]" required />
</p>

<p>
    <label for="metodoPago">Selecciona método de pago</label>
    <select id="metodoPago" name="contacto[metodosPago][]" multiple size="3">
        <option value="VISA" selected>VISA</option>
        <option value="Mastercard">Mastercard</option>
        <option value="Paypal">Paypal</option>
    </select>
</p>

<p>
    <input type="checkbox" id="marcar" name="contacto[marcar]"
           value="1" required />
    <label for="marcar">Aceptar los términos y condiciones</label>
</p>

<input type="submit" value="Enviar" />
</form>
```

Fichero: `Views/contacto/resultado.php`

```
<h1>Datos recibidos</h1>

<p><strong>Nombre:</strong> <?= e($contacto->nombre) ?></p>
<p><strong>Apellidos:</strong>
    <?= e($contacto->apellidos) ?: 'No indicados' ?>
</p>
<p><strong>Correo electrónico:</strong> <?= e($contacto->correo) ?></p>
<p><strong>Métodos de pago:</strong>
    <?= e(implode(', ', $contacto->metodosPago)) ?: 'Ninguno' ?>
</p>
<p><strong>Términos y condiciones:</strong>
    <?= $contacto->marcar ? 'aceptados' : 'no aceptados' ?>
</p>

<p>
    <a href="index.php?controller=contacto&amp;action=formulario">
        Volver al formulario </a>
</p>
```

> **Atención**
> La vista **no decide nada**: no consulta `$_POST` ni llama al modelo. Solo recibe variables ya preparadas (`$contacto`) y las muestra, siempre escapadas con `e()`.

3.7. Qué ocurre al recargar la página

Cuando se envía el formulario, `procesar()` muestra el resultado directamente, como respuesta al propio POST. Si el usuario recarga la página, el navegador le pide confirmar el reenvío del formulario (apartado 1.3) y, si acepta, se procesa de nuevo la misma petición. En esta aplicación no ocurre nada grave porque no se guarda nada, pero en una aplicación que almacenara los datos el contacto quedaría **duplicado**.

La solución estándar es el patrón ***Post/Redirect/Get*** (PRG):

- El navegador envía los datos con **POST**.
- El controlador los procesa y responde con una redirección (`header('Location: ...')` y `exit`, apartado 5.2).
- El navegador pide la nueva página con **GET**, y recargarla solo repite un GET inocuo.

> **Nota**
> **¿Por qué no lo aplicamos aquí?** Después de la redirección, la nueva petición GET ya no tiene los datos del POST. Para mostrarlos haría falta conservarlos entre las dos peticiones (en una sesión, un fichero o una base de datos), y todavía no los hemos visto. Cuando guardemos datos retomaremos este patrón.

4. Validación y saneamiento de formularios

La arquitectura de seguridad óptima exige una **doble validación**: una en el *front-end* (HTML, JavaScript), que mejora la experiencia del usuario, y otra obligatoria e ineludible en el *back-end*, porque las peticiones del cliente pueden alterarse con facilidad.

Debemos realizar dos procedimientos:

- **Saneamiento:** elimina caracteres inapropiados o potencialmente peligrosos de la cadena recibida.
- **Validación:** verifica de forma lógica si el dato cumple el patrón requerido (si es un número, un email real, etc.).

PHP dispone de una extensión que facilita ambos procesos. Se llama Filter y está activa por defecto. La función principal es filter_var(), que filtra una variable con el filtro indicado y sirve tanto para sanear como para validar.

- Los **filtros de saneamiento** (`FILTER_SANITIZE_*`) eliminan los caracteres que no se ajustan al tipo de filtro.
- Una vez limpia la información, los **filtros de validación** (`FILTER_VALIDATE_*`) comprueban que cumple los requisitos: desde que el campo exista hasta que sea numérico o un correo correcto.

Ejemplo en el que nos aseguramos de que la información recibida es un email:

```php
$email = trim($_POST['email'] ?? '');

// Primero saneamos: se eliminan los caracteres no válidos en un correo
$emailSaneado = filter_var($email, FILTER_SANITIZE_EMAIL);

// Después validamos que la cadena saneada tenga la estructura de un correo real
if (filter_var($emailSaneado, FILTER_VALIDATE_EMAIL) === false) {
    echo 'ERROR: ' . htmlspecialchars($email) . ' NO es una dirección de email válida';
}

// Podemos añadir los controles que consideremos oportunos (p. ej., un dominio concreto)
```

4.1. Opciones de filtrado y banderas (flags)

Los filtros básicos pueden quedarse cortos ante las necesidades de una aplicación real. Para afinar su comportamiento, `filter_var()` admite un tercer parámetro opcional: las **banderas** (*flags*).

Un caso crítico son los números con decimales (precios, distancias, pesos…). Si aplicamos el saneamiento de números flotantes sin más, PHP elimina de la cadena los puntos y las comas y rompe por completo el valor. Para que el número conserve su punto decimal hay que activar `FILTER_FLAG_ALLOW_FRACTION`.

```php
<?php
// El usuario introduce un precio con decimales y caracteres erróneos
$precioOriginal = ' $145.99 € ';

// Saneamiento básico (INCORRECTO para decimales): elimina también el punto
$precioIncorrecto = filter_var($precioOriginal, FILTER_SANITIZE_NUMBER_FLOAT);
echo $precioIncorrecto;   // 14599  (¡el precio se ha multiplicado por 100!)

// Saneamiento con flag (CORRECTO): se respeta el punto de la fracción
$precioSaneado = filter_var(
    $precioOriginal,
    FILTER_SANITIZE_NUMBER_FLOAT,
    FILTER_FLAG_ALLOW_FRACTION
);
echo $precioSaneado;      // 145.99

// Validación: una vez saneado, comprobamos si es un float válido
if (filter_var($precioSaneado, FILTER_VALIDATE_FLOAT) !== false) {
    echo 'El precio es un número decimal válido.';
} else {
    echo 'Error: El precio no es numérico.';
}
```

4.2. Filtros de validación más usados

Filtro	Qué comprueba	Si es válido devuelve
`FILTER_VALIDATE_INT`	Un entero (admite `min_range` y `max_range`)	El entero, o `false`
`FILTER_VALIDATE_FLOAT`	Un número decimal	El `float`, o `false`

Filtro	Qué comprueba	Si es válido devuelve
`FILTER_VALIDATE_EMAIL`	Un correo electrónico	La cadena, o `false`
`FILTER_VALIDATE_URL`	Una URL	La cadena, o `false`
`FILTER_VALIDATE_IP`	Una dirección IPv4 o IPv6	La cadena, o `false`
`FILTER_VALIDATE_BOOLEAN`	`true`/`false`, `1`/`0`, `yes`/`no`, `on`/`off`	Un `bool` (`null` con `FILTER_NULL_ON_FAILURE` si no se reconoce)
`FILTER_VALIDATE_REGEXP`	Que cumpla una expresión regular	La cadena, o `false`

```php
// Edad: entero entre 18 y 120
$edad = filter_var(
    $_POST['edad'] ?? '',
    FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 18, 'max_range' => 120]]
);

if ($edad === false) {
    echo 'La edad debe ser un número entre 18 y 120.';
}

// Código con formato concreto: dos letras mayúsculas y dos dígitos (AB12)
$codigo = filter_var(
    'AB12',
    FILTER_VALIDATE_REGEXP,
    ['options' => ['regexp' => '/^[A-Z]{2}\d{2}$/']]
);
```

> **Atención**
> Compara siempre con `=== false`. `filter_var('0', FILTER_VALIDATE_INT)` devuelve `0`, que es un valor válido pero «falso» en una condición: `if (!$edad)` lo trataría como error.

filter_input()

`filter_input()` hace lo mismo que `filter_var()` pero leyendo directamente de la petición (`INPUT_GET`, `INPUT_POST`, `INPUT_COOKIE`, `INPUT_SERVER`). Devuelve `null` si el campo no existe y `false` si existe pero no es válido.

```php
$edad = filter_input(
    INPUT_POST,
    'edad',
    FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 18, 'max_range' => 120]]
);

if ($edad === null) {
    echo 'Falta el campo edad';
} elseif ($edad === false) {
    echo 'Edad no válida';
}
```

4.3. Filtros de saneamiento

Filtro	Efecto
`FILTER_SANITIZE_EMAIL`	Elimina todos los caracteres no permitidos en una dirección de correo.
`FILTER_SANITIZE_NUMBER_INT`	Deja solo dígitos y los signos `+` y `-`.
`FILTER_SANITIZE_NUMBER_FLOAT`	Igual que el anterior; con flags (`FILTER_FLAG_ALLOW_FRACTION`, …) conserva el punto, la coma o la `e`.
`FILTER_SANITIZE_URL`	Elimina los caracteres no permitidos en una URL.
`FILTER_SANITIZE_FULL_SPECIAL_CHARS`	Convierte los caracteres especiales de HTML en entidades (similar a `htmlspecialchars()`).

> **Atención**
> `FILTER_SANITIZE_STRING` está **obsoleto desde PHP 8.1**. No lo uses: para mostrar texto en HTML emplea `htmlspecialchars()`.

4.4. Escapar la salida: htmlspecialchars()

Sanear y validar al **recibir** los datos no basta: además hay que **escapar** cualquier dato del usuario al **mostrarlo** en HTML. Es la defensa contra los ataques XSS (*cross-site scripting*), en los que un usuario introduce código JavaScript que se ejecutaría en el navegador de otros visitantes.

```php
$comentario = '<script>alert("hack")</script>';

echo $comentario;                          // PELIGRO: el navegador ejecutaría el script
echo htmlspecialchars($comentario);        // se muestra como texto, sin ejecutarse
```

RESULTADO DE HTMLSPECIALCHARS()

```
&lt;script&gt;alert(&quot;hack&quot;)&lt;/script&gt;
```

> **Nota**
> Desde PHP 8.1 `htmlspecialchars()` convierte por defecto las comillas simples y dobles. Aun así es buena práctica indicar los parámetros explícitamente: `htmlspecialchars($texto, ENT_QUOTES, 'UTF-8')`.

4.5. Validar y volver a mostrar el formulario en MVC

Aplicamos lo visto al formulario de contacto del apartado 3. Cuando los datos no son válidos hay que volver a mostrar el formulario con un mensaje de error junto a cada campo y **con lo que el usuario ya había escrito**, para que solo tenga que corregir lo necesario.

En MVC cada clase tiene una única responsabilidad: el modelo representa los datos y el controlador reparte el trabajo, así que las reglas de validación no deben ir en ninguno de los dos. Haremos lo que hacen frameworks como Laravel con sus *Form Requests*: una clase por

formulario, en `src/Requests`, que recibe el **array que llega de `$_POST`**, lo **sanea** y lo **valida**. El modelo `Contacto` solo se crea cuando los datos ya son válidos.

```
src/
├── Controllers/
│   └── ContactoController.php
├── Lib/
│
└── Requests/                    ← carpeta nueva
    └── ContactoRequest.php      ← sanea y valida el array recibido
Views/
└── contacto/
    └── formulario.php
```

El recorrido completo es este:

- El formulario se envía por **POST** a `procesar`. El controlador pasa el array `contacto[...]` a `ContactoRequest`, que lo sanea y lo valida.
- Si hay **errores, redirige** a la acción `formulario` y muestra los errores y los datos anteriores.
- Si los datos son **válidos**, se crea el `Contacto` y se muestra el resultado. Todavía no hay dónde guardarlo, así que se muestra directamente.

1. La Request. Recibe el array, lo sanea en `sanear()` (`trim()` de los textos, `FILTER_SANITIZE_EMAIL` para el correo, solo valores de tipo texto) y lo valida en `validar()` con las reglas del apartado 4.2. Ofrece tres métodos: `esValido()`, `errores()` y `datos()`. El controlador solo pregunta si los datos valen; no sabe **cómo** se comprueban.

Fichero: src/Requests/ContactoRequest.php

```php
<?php
declare(strict_types=1);

namespace Requests;

final class ContactoRequest
{
    /** Métodos de pago que ofrece el formulario */
    public const array METODOS_PAGO = ['VISA', 'Mastercard', 'Paypal'];

    /** @var array<string, mixed> datos ya saneados */
    private array $datos = [];

    /** @var array<string, string> mensajes de error, indexados por campo */
    private array $errores = [];

    /** @param array<string, mixed> $entrada el array contacto[...] recibido */
    public function __construct(array $entrada)
    {
        $this->sanear($entrada);
        $this->validar();
    }

    public function esValido(): bool
    {
        return $this->errores === [];
    }

    /** @return array<string, string> */
    public function errores(): array
    {
        return $this->errores;
    }

    /** @return array<string, mixed> los datos saneados */
    public function datos(): array
    {
        return $this->datos;
    }

    private function sanear(array $entrada): void
    {
        // El usuario puede manipular la petición: solo aceptamos textos
        $texto = static fn(mixed $v): string => is_string($v) ? trim(strip_tags($v)) : '';

        $this->datos = [
             'nombre'           => $texto($entrada['nombre'] ?? ''),
             'apellidos'        => $texto($entrada['apellidos'] ?? ''),
             'correo'           => trim($entrada['correo'] ?? ''),
             'direccion'        => $texto($entrada['direccion'] ?? ''),
             'telefono'         => $texto($entrada['telefono'] ?? ''),
             'fecha_nacimiento' => $texto($entrada['fecha_nacimiento'] ?? ''),
             'marcar'           => isset($entrada['marcar']) ? '1' : '',
        ];


    }

     private function validar(): void
     {
         $d = $this->datos;

         // --- Nombre (obligatorio, 2-50 caracteres) ---
         if ($d['nombre'] === '') {
```

```
            $this->errores['nombre'] = "El nombre es obligatorio.";
        } elseif (mb_strlen($d['nombre']) < 2) {
            $this->errores['nombre'] = "El nombre debe tener al menos 2 caracteres.";
        } elseif (mb_strlen($d['nombre']) > 50) {
            $this->errores['nombre'] = "El nombre no puede superar los 50 caracteres.";
        } elseif (!preg_match('/^[\pL\s\-\'\.]+$/u', $d['nombre'])) {
            $this->errores['nombre'] = "El nombre solo puede contener letras, espacios, guiones y
    puntos.";
        }

        // --- Apellidos (opcional, 2-100 caracteres si se indica) ---
        if ($d['apellidos'] !== '') {
            if (mb_strlen($d['apellidos']) < 2) {
                $this->errores['apellidos'] = "Los apellidos deben tener al menos 2 caracteres.";
            } elseif (mb_strlen($d['apellidos']) > 100) {
                $this->errores['apellidos'] = "Los apellidos no pueden superar los 100
    caracteres.";
            } elseif (!preg_match('/^[\pL\s\-\'\.]+$/u', $d['apellidos'])) {
                $this->errores['apellidos'] = "Los apellidos solo pueden contener letras,
    espacios, guiones y puntos.";
            }
        }

        // --- Correo (obligatorio, formato válido, máx. 255 caracteres) ---
        if ($d['correo'] === '') {
            $this->errores['correo'] = "El correo es obligatorio.";
        } elseif (mb_strlen($d['correo']) > 255) {
            $this->errores['correo'] = "El correo no puede superar los 255 caracteres.";
        } elseif (!filter_var($d['correo'], FILTER_VALIDATE_EMAIL)) {
            $this->errores['correo'] = "El formato del correo no es válido.";
        }

        // --- Dirección (opcional, máx. 255 caracteres) ---
        if ($d['direccion'] !== '' && mb_strlen($d['direccion']) > 255) {
            $this->errores['direccion'] = "La dirección no puede superar los 255 caracteres.";
        }

        // --- Teléfono (opcional, 9-15 dígitos, puede incluir +, espacios o guiones) ---
        if ($d['telefono'] !== '' && !preg_match('/^\+?[0-9\s\-]{9,15}$/', $d['telefono'])) {
            $this->errores['telefono'] = "El teléfono debe tener entre 9 y 15 dígitos (puede
    incluir +, espacios o guiones).";
        }

        // --- Fecha de nacimiento(opcional, formato Y-m-d, o futura, no anterior a 1900) ---
        if ($d['fecha_nacimiento'] !== '') {
            $fecha = DateTime::createFromFormat('Y-m-d', $d['fecha_nacimiento']);
            if (!$fecha || $fecha->format('Y-m-d') !== $d['fecha_nacimiento']) {
                $this->errores['fecha_nacimiento'] = "La fecha de nacimiento no es válida.
    Formato esperado: AAAA-MM-DD.";
            } elseif ($fecha > new DateTime('today')) {
                $this->errores['fecha_nacimiento'] = "La fecha de nacimiento no puede ser
    futura.";
            } elseif ($fecha < new DateTime('1900-01-01')) {
                $this->errores['fecha_nacimiento'] = "La fecha de nacimiento no puede ser anterior
    a 1900.";
            }
        }

        // --- Términos y condiciones ---
        if ($d['marcar'] !== '1') {
            $this->errores['marcar'] = "Debes aceptar los términos y condiciones.";
        }
    }
}
```

3. El controlador. En `procesar()` se crea la Request con el array recibido (si alguien manipula la petición y `contacto` no es un array, se usa uno vacío). Si no es válida, se vuelve a mostrar el formulario con los errores.

Fichero: `src/Controllers/ContactoController.php`

```php
<?php
declare(strict_types=1);

namespace Controllers;

use Lib\Pages;
use Models\Contacto;
use Requests\ContactoRequest;

final class ContactoController
{
    private Pages $pagina;

    public function __construct()
    {
        $this->pagina = new Pages();
    }

    /** Muestra el formulario inicial por primera vez (GET) */
    public function formulario(): void
    {
        $this->pagina->render('contacto/formulario', [
            'contacto' => new Contacto(metodosPago: ['VISA']),
            'metodos'  => ContactoRequest::METODOS_PAGO,
            'errores'  => [],
        ]);
    }

    /** Procesa el formulario (POST) sin usar sesiones ni redirecciones */
    public function procesar(): void
    {
        if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
            $this->formulario();
            return;
        }

        $entrada = $_POST['contacto'] ?? [];
        $request = new ContactoRequest(is_array($entrada) ? $entrada : []);

        // Si los datos NO son válidos, renderizamos directamente el formulario con los datos
recibidos y los errores
        if (!$request->esValido()) {
            $this->pagina->render('contacto/formulario', [
                'contacto' => Contacto::fromArray($request->datos()),
                'metodos'  => ContactoRequest::METODOS_PAGO,
                'errores'  => $request->errores(),
            ]);
            return;
        }

        // Si los datos son válidos, renderizamos la vista del resultado
        $contacto = Contacto::fromArray($request->datos());
        $this->pagina->render('contacto/resultado', ['contacto' => $contacto]);
    }
}
```

4. La vista. Cada campo recupera su valor del objeto `$contacto` (creado con los datos anteriores) y muestra su error si existe. El `select` múltiple marca como seleccionados los métodos que el usuario había elegido, y la casilla recupera su estado con `checked`:

Fichero: `Views/contacto/formulario.php`

```
<h2>Nuevo contacto</h2>
<form method="POST" action="index.php?controller=contacto&amp;action=procesar" novalidate>
    <p>
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="contacto[nombre]" value="<?= e($contacto->nombre)
?>" />
        <?php if (isset($errores['nombre'])): ?>
            <span class="error" style="color: red;"><?= e($errores['nombre']) ?></span>
        <?php endif; ?>
    </p>

    <p>
        <label for="apellidos">Apellidos</label>
        <input type="text" id="apellidos" name="contacto[apellidos]" value="<?= e($contacto-
>apellidos) ?>" />
        <?php if (isset($errores['apellidos'])): ?>
            <span class="error" style="color: red;"><?= e($errores['apellidos']) ?></span>
        <?php endif; ?>
    </p>

    <p>
        <label for="correo">Correo electrónico</label>
        <input type="email" id="correo" name="contacto[correo]" value="<?= e($contacto-
>correo) ?>" />
        <?php if (isset($errores['correo'])): ?>
            <span class="error" style="color: red;"><?= e($errores['correo']) ?></span>
        <?php endif; ?>
    </p>

    <p>
        <label for="metodoPago">Selecciona método de pago</label>
        <select id="metodoPago" name="contacto[metodosPago][]" multiple size="3">
            <?php foreach ($metodos as $metodo): ?>
                <?php $sel = in_array($metodo, $contacto->metodosPago, true); ?>
                <option value="<?= e($metodo) ?>" <?= $sel ? 'selected' : '' ?>>
                    <?= e($metodo) ?>
                </option>
            <?php endforeach; ?>
        </select>
        <?php if (isset($errores['metodosPago'])): ?>
            <span class="error" style="color: red;"><?= e($errores['metodosPago']) ?></span>
        <?php endif; ?>
    </p>

    <p>
        <input type="checkbox" id="marcar" name="contacto[marcar]" value="1" <?= $contacto-
>marcar ? 'checked' : '' ?> />
        <label for="marcar">Aceptar los términos y condiciones</label>
        <?php if (isset($errores['marcar'])): ?>
            <span class="error" style="color: red;"><?= e($errores['marcar']) ?></span>
        <?php endif; ?>
    </p>

    <input type="submit" value="Enviar" />
</form>
```

Nota

¿Por qué VISA se preselecciona en el controlador y no en el HTML? Porque `formulario()` la usa como valor por defecto solo cuando **no hay** datos de un envío anterior. Si estuviera fija en la vista, volvería a aparecer seleccionada aunque el usuario la hubiera deseleccionado.

Atención

Todo dato que se vuelque en un atributo (`value="..."`) debe escaparse con `e()` (apartado 4.4). Si no, un valor como `" onfocus="alert(1)` rompería el atributo y permitiría inyectar código (XSS).

Consejo

Para probar la validación del servidor, añade `novalidate` a la etiqueta `<form>` de forma temporal. Si no, el navegador bloquea el envío con `required` y `minlength` y la petición no llega a PHP.